Архітектура пошукової системи актів Вищої ради правосуддя на Elixir

Анотація

У статті запропоновано архітектуру чистої (clean-room) серверної пошукової системи актів Вищої ради правосуддя. Система побудована на Elixir (Bandit + Plug), PostgreSQL як первинному джерелі даних, OpenSearch як пошуковому індексі та HTMX для прогресивного покращення інтерфейсу. Описано поділ відповідальності між трьома сервісами: веб-застосунком, сервісом повної індексації та сервісом інкрементальних оновлень. Показано чотири типові сценарії розгортання та ключові принципи мінімізації залежностей, збереження стану пошуку в URL і повного серверного рендерингу HTML.

1 Вступ

Сучасні публічні реєстри правових актів вимагають поєднання повнотекстового, параметричного та семантичного пошуку при збереженні високої прозорості архітектури та мінімуму зовнішніх залежностей. Традиційні підходи, що спираються на односторінкові застосунки та великі клієнтські бандли, ускладнюють аудит, підвищують вартість супроводу та ускладнюють відтворення стану пошуку через URL.

Метою роботи є побудова чистої серверної архітектури пошукової системи актів Вищої ради правосуддя (ВРП), яка:

- використовує PostgreSQL як єдине первинне джерело правди;
- застосовує OpenSearch виключно для читання та ранжування;
- генерує HTML на сервері з підтримкою HTMX;
- зберігає повний стан пошуку в адресному рядку;
- розділяє відповідальність між веб-застосунком та окремими сервісами індексації.

Система орієнтована на дані відкритих наборів ВРП (рішення з поіменним голосуванням, акти, розподіл справ, стягнення, відсторонення тощо) і підтримує як класичний повнотекстовий пошук, так і гібридний режим із семантичною складовою.

2 Типова архітектура

Типова архітектура чистої серверної пошукової системи складається з чотирьох логічних шарів.

2.1 Первинне сховище

Реляційна база даних (PostgreSQL) зберігає нормалізовані записи актів, учасників, голосувань і зв'язків. Усі зміни даних відбуваються виключно тут. Пошуковий індекс ніколи не є джерелом правди.

2.2 Пошуковий індекс

OpenSearch (або сумісний рушій) містить денормалізовані документи з повнотекстовими полями, вкладеними об'єктами та векторними представленнями. Індекс оновлюється асинхронно і використовується лише для читання.

2.3 Сервіси індексації

Виділяються два незалежні сервіси:

- сервіс повної (первинної) індексації — потокове читання з PostgreSQL і пакетне завантаження в OpenSearch;
- сервіс інкрементальних оновлень — реакція на зміни в первинній базі (через LISTEN/NOTIFY, outbox або опитування) і точкове оновлення індексу.

2.4 Веб-шар

Легкий HTTP-сервер (Bandit) з маршрутизацією на Plug приймає запити, будує запити до OpenSearch, формує HTML-сторінки та JSON-відповіді. Інтерактивність забезпечується HTMX без окремого клієнтського фреймворку. Стан пошуку (запит, фільтри, сторінка, режим) повністю відображається в URL.

Такий поділ забезпечує:

- незалежне масштабування веб-шару та індексації;
- можливість повного переіндексування без зупинки публічного сервісу (через alias);
- мінімальну поверхню атаки (веб-сервіс має лише права читання індексу).

3 Архітектура системи ВРП

3.1 Загальна схема

Система ВРП реалізує описану типову архітектуру у вигляді трьох Elixir-застосунків:

1. `vrp-web` — публічний веб-сервіс (Bandit + Plug);
2. `vrp-indexer` — CLI-сервіс повної індексації;
3. `vrp-updater` — довгоживучий сервіс інкрементальних оновлень.

PostgreSQL виступає первинним сховищем. Дані надходять із зовнішніх процесів обробки відкритих наборів і записуються безпосередньо в реляційні таблиці. OpenSearch використовує alias `vrp_acts`; перемикання індексу відбувається атомарно після успішної повної індексації.

3.2 Веб-сервіс (`vrp-web`)

Веб-сервіс є повністю безстановим. Він виконує такі функції:

- розбір параметрів запиту з URL;
- побудову гібридного запиту до OpenSearch (BM25 + k-NN з pre-filter);
- формування карток видачі, фасетів і повного тексту акта;
- генерацію HTML 4.01 / XHTML з фрагментами для HTMX;
- надання JSON API з ідентичним контрактом даних.

Основний пошук реалізовано через звичайну HTML-форму методу GET, що гарантує можливість пересилання посилань. HTMX застосовується лише для автодоповнення, підвантаження додаткових результатів і оновлення фасетів без повного перезавантаження сторінки.

3.3 Сервіс повної індексації (`vrp-indexer`)

Сервіс працює в чотири стадії:

1. підключення до PostgreSQL і оцінка обсягу даних;
2. потокове читання записів із необхідними JOIN;
3. пакетне (bulk) відправлення документів в OpenSearch;
4. оновлення alias і фінальна статистика.

Такий підхід дозволяє індексувати десятки тисяч актів із вкладеними структурами (учасники, поіменне голосування, файли) без утримання всього набору в пам'яті.

3.4 Сервіс інкрементальних оновлень (`vrp-updater`)

Сервіс відстежує зміни в PostgreSQL. Рекомендований механізм — таблиця outbox або тригери з `NOTIFY`. При зміні акта, пов'язаних учасників чи голосування сервіс формує актуальний документ і виконує індексацію або видалення відповідного документа в OpenSearch. Це забезпечує близьку до реального часу актуальність пошукової видачі.

3.5 Модель даних і контракт індексу

Документи в OpenSearch зберігаються з числовими ключами полів. Вкладені списки (учасники, голосування, файли) мають тип `nested`. Текстові поля використовують український аналізатор. Векторні представлення фрагментів тексту підтримують семантичний пошук. Версія mapping фіксується в метаданих індексу і перевіряється при старті веб-сервісу.

3.6 Сценарії розгортання

Система підтримує чотири основні сценарії:

- повністю локальний стенд із синтетичними даними;
- гібридний режим (віддалений PostgreSQL + локальний OpenSearch);
- хмарний режим, у якому веб-сервіс підключається до зовнішніх кластерів;
- повністю контейнеризований стек через Docker Compose.

Усі сценарії конфігуруються через змінні середовища і не вимагають змін коду.

4 Висновки

Запропонована архітектура забезпечує чіткий поділ відповідальності між первинним сховищем, пошуковим індексом і веб-шаром. Використання Elixir з Bandit і Plug дозволяє отримати високопродуктивний серверний рендеринг при мінімальній кількості залежностей. Розділення повної та інкрементальної індексації дає змогу підтримувати актуальність даних без простою публічного сервісу. Підхід є придатним для побудови відкритих реєстрів правових актів, що вимагають прозорості, відтворюваності стану пошуку та можливості незалежного масштабування компонентів.